Doc. no.: P0492R1
Date: 2017-02-06
Reply to: Beman Dawes <bdawes at acm dot org>
S. Davis Herring <herring at lanl dot gov>
Nicolai Josuttis <nico at josuttis dot de>
Jason Liu <jasonliu dot development at gmail dot com>
Billy O'Neal <bion at microsoft dot com>
P.J. Plauger <pjp at dinkumware dot com>
Jonathan Wakely <cxx at kayari dot org>
Audience: Library

Proposed Resolution of C++17 National Body Comments for Filesystems(R1)

This document proposes resolutions of C++17 CD National Body Comments from P0488R0 related to Filesystems (27.10 [filesystems]). Some "Late" Comments from P0489R0 are also considered.

The proposed resolutions in this paper represent the consensus of the Library Working Group's Filesystem small group (SG) unless otherwise indicated.

Proposed wording is relative to the C++ working paper, N4618.

Revision History

P0492R1 Pre-Kona mailing

- Direction remains unchanged.
- Updated WP base document to N4618.
- Modifications to reflect editorial changes to the standard WP. Most importantly, the name of the class `path` exposition only data member was changed from `pathname` to `pathstring`.
- Proposed wording supplied for numerous NB comments that had no proposed wording.
- Numerous minor and editorial corrections.

P0492R0 Post-Issaquah mailing

The initial version of this document.

Markup

Notes about work that remains to be done, who is planning to do that work, or wording that requires specially careful review by the LWG. If the note applies to proposed changes to the working paper, the changes may not yet ready to be voted into the C++17 working paper.

SG informative notes that will remain in this proposal but do not become part of the WP.

Instructions to the project editor.

Text to be removed from the WP.

Text to be added to the WP.

NB Technical and general comments

Comments related to `filename()`

These comments are tightly coupled and best resolved as a whole.

US 25; 27.10.8.4.10 [path.query]: `has_filename()` is equivalent to just `!empty()`
Status: Reject

Resolved by US-52, US-53, US-54, and US-60.

US 37; 27.10.4.12 [fs.def.normal.form]: The term “redundant current directory (dot) elements” is not defined
Status: Accept with modifications

Discussion

Invariants:

1. An empty path remains empty, a non-empty path remains non-empty.
2. An absolute path remains absolute, a relative path remains relative.
3. The normal form has a trailing *directory-separator* if and only if the original path had a trailing *directory-separator*.

4. No redundant *dot*, *dot-dot*, or *directory-separator*s.
5. *directory-separator*s are in the *preferred-separator* form.

Desired behavior:

Case #	**Generic format pathname**	**After normalization**	Nico's 4 Feb email
1	`""`	`""`	
2	`"."`	`"."`	
3	`".."`	`".."`	
4	`"/"`	`"/"`	
5	`"//"`	`"//"`	
6	`"/foo"`	`"/foo"`	
7	`"/foo/"`	`"/foo/"`	
8	`"/foo/."`	`"/foo"`	`"/foo/"`
9	`"/foo/bar/.."`	`"/foo"`	`"/foo/"`
10	`"/foo/.."`	`"/"`	
11	`"/."`	`"/"`	
12	`"/./"`	`"/./"`	`"/"`
13	`"/./."`	`"/"`	
14	`"/././"`	`"/./"`	`"/"`
15	`"/./././"`	`"/"`	
16	`"./"`	`"./"`	
17	`"./."`	`"."`	
18	`"./././"`	`"./"`	
19	`"./././."`	`"."`	
20	`"./././/"`	`"./"`	
21	`"./././/."`	`"."`	
22	`"foo/.."`	`"."`	
23	`"foo/../"`	`"./"`	
24	`"foo/../.."`	`".."`	
25 Windows	`"C:bar/.."`	`"C:."`	
26 Windows	`"C:"`	`"C:"`	
27	`"//host/bar/.."`	`"//host/"`	
28	`"//host"`	`"//host"`	
29	`"foo/../foo/.."`	`"."`	
30	`"foo/../foo/../.."`	`".."`	
31	`"../foo/../foo/.."`	`".."`	
32	`"../.f/../f"`	`"../f"`	
33	`"../f/../.f"`	`"../.f"`	
34	`"../../../."`	`"../.."`	
35	`"../../.././"`	`"../../"`	

Beman and Nico need to resolve their different suggested normalizations for cases 8, 9, 12, and 14. This is probably just a

synchronization issue rather than a controversy. They are in total agreement about invariants and are just trying to settle on the best wording to describe the desired results.

Change 27.10.4.12 [fs.def.normal.form]:

> **normal form**
> A path with no redundant current directory (*dot*) elements, no redundant parent directory (*dot-dot*) elements, and no redundant *directory-separators*. The normal form for an empty path is an empty path. The normal form for a path ending in a *directory-separator* that is not the root directory has a current directory (*dot*) element appended. A path in normal form is said to be *normalized*. The process of obtaining a normalized path from a path that is not in normal form is called *normalization*. [*Note:* The rule that appends a current directory (*dot*) element supports operating systems like OpenVMS that use different syntax for directory names and regular file names. —*end note*]

Option A

A generic format pathname is in normal form if empty or has:

- Only single *directory-separator* sequences or a leading sequence of two *directory-separator*s.
- No *dot* filenames except:
  - if the pathname would otherwise be empty, the pathname consists of a *dot* filename .
  - if the *relative-path* portion of the pathname would otherwise consist solely of a *director-separator, the relative-path* portion of the pathname consists of a *dot* filename followed by a *preferred-separator*.
- No sequences of *name directory-separator dot-dot*, applied recursively.
- Only the *preferred-separator* form of *directory-separator*s.

A path in normal form is said to be *normalized*. The process of obtaining a normalized path from a path that is not in normal form is called *normalization*.

Option B

For a non-empty generic format pathname:

- *dot* filenames are removed, except when removal would result in an empty pathname or change a relative pathname into an absolute pathname.

  [*Example:* Normal form of `"./././"` is `"./"` on POSIX —*end example*]

- Sequences of *name directory-separator dot-dot* are recursively removed. If that would result in a trailing *relative-path directory-separator* becoming the first *relative-path* character, a *dot* filename is first inserted at the beginning of the *relative-path*.

  [*Example:* Normal form of `"/foo/../"` is `"/./"` on POSIX —*end example*]

- Sequences of multiple *directory-separator*s, except two leading *directory-separator*s, are replaced by a single *preferred-separator*.

  [*Example:* Normal form of `"//foo///bar"` is `"//foo/bar"` on POSIX —*end example*]

- *slash* directory-separators are replaced by *preferred-separator*s.

  [*Example:* Normal form of `"//foo/bar"` is `"\\\\foo\\bar"` on Windows —*end example*]

A path in normal form is said to be *normalized*. The process of obtaining a normalized path from a path that is not in normal form is called *normalization*.

Add the following note as a new paragraph at the end of [path.generic]:

> *Normal form and several other aspects of the class path spec are hard to understand without an*

understanding of the role of a trailing directory-separator. Yet even experts are not always aware of the following. Thus the added note.

[*Note:* The *relative-path* portion of a generic format pathname may end with a *directory-separator*. Its use is:

- Required for operating systems such as z/OS and OpenVMS where native path syntax for directories is different from non-directory native path syntax, and thus directory paths must be lexically distinguished from non-directory paths.
- Required by POSIX utilities for tool dependent purposes such as indicating that a symlink is not to be followed.
- Required by some programs to distinguish directory pathnames that are to be handled in some special way.
- Recommended by some coding guidelines to clearly distinguish directory names from non-directory names.
- Recommended occasionally to ensure that an error is reported rather than a file overwritten if a regular file pathname is passed mistakenly as an argument when a directory pathname was intended.

—end note]

Beman supplied the above wording. It has not been reviewed by the small group.

US 51; 27.10.8.4.3 [path.append] ¶2.3: Failing to add / when appending empty string prevents useful apps
Status: Reject

Resolved by US-74/CA-3

US 52; 27.10.8.4.5 [path.modifiers] ¶5: `remove_filename()` postcondition is not by itself a definition
Status: Accept with modifications

Also see LWG LWG 2665.

Change 27.10.8.4.5 [path.modifiers]:

```
path& remove_filename();
```

Postcondition: `!has_filename()`.

Effects: If `has_filename()` then remove the minimum number of trailing characters from generic format pathname such that there is one less element in `[begin(), end())`. Otherwise, no effect.

Returns: `*this`.

[*Example:*

```
std::cout << path("/foo").remove_filename(); // outputs "/"
std::cout << path("/").remove_filename();    // outputs "/"
```

—end example]

US 53; 27.10.8.4.5 [path.modifiers] ¶4 (was ¶7): `remove_filename()`'s name does not correspond to its behavior
Status: Accept with modifications

As a consequence of the US-52 and US-60 fixes, the name now corresponds to the behavior.

US 54; 27.10.8.4.5 [path.modifiers] ¶7 (was ¶10): `replace_filename()` is broken
Status: Accept with modifications

See US-52 and US-60 fixes.

Change 27.10.8.4.5 [path.modifiers]:

```
path& replace_filename(const path& replacement);
```

Effects: Equivalent to:

```
if (has_filename()) {
  remove_filename();
  operator/=(replacement);
}
```

Returns: `*this`.

[*Example:*

```
std::cout << path("/foo").replace_filename("bar");  // outputs "/bar"
std::cout << path("/").replace_filename("bar");     // outputs "bar"
```

—*end example*]

Change 27.10.12.2 [directory_entry.mods]:

```
void replace_filename(const path& p);
```

Postconditions: `path() == x.parent_path() / p` `x.replace_filename(p)` where `x` is the value of `path()` before the function is called.

Beman Dawes supplied the above wording. It has not been reviewed by the small group.

US 60; 27.10.8.4.9 [path.decompose] ¶6: `path("/foo/").filename()==path(".")` is surprising
Status: Accept with modifications

Discussion:

The `filename()` behavior at issue is specified in 27.10.8.4.9 [path.decompose] ¶6:

Returns: `empty() ? path() : *--end()`.

So the underlying problem is in 27.10.8.5 path iterators [path.itr] ¶4:

4 For the elements of pathname in the generic format, the forward traversal order is as follows:

(4.1) — The *root-name* element, if present.

(4.2) — The *root-directory* element, if present. [*Note:* the generic format is required to ensure lexicographical comparison works correctly. —*end note*]

(4.3) — Each successive *filename* element, if present.

(4.4) — *dot*, if one or more trailing non-root *slash* characters are present.

The ¶4.4 iterator behavior has always been surprising and controversial, so the LWG SG looked at the possible ways of dealing with trailing non-root slash characters:

dot, if one or more trailing non-root *slash* characters are present.

> Not acceptable because it loses the fact that the path represents a directory, yet knowing that is required for some operating systems (e.g. POSIX, z/OS, OpenMVS). We want to ensure that reconstruction by iteration yields the same path as the original for definitions of "same".

dot *slash*, if one or more trailing non-root *slash* characters are present.

> Not acceptable because a slash is semantically ambiguous; it is already used to indicate a root, so also using it to indicate a directory path would be a trap all too easy to fall into.

dot an empty element, if one or more trailing non-root *slash* characters are present.

> This works, and was discussed at length by the SG. It appears to resolve a number of concerns raised by other NB comments and by the SG during discussion of those comments. All the ripple effects of this change need to be identified, and wording changes proposed where needed or desirable.
>
> Because this is a design change, Nico presented it to LEWG as part of "Clarify filename". It was accepted by a vote of 13/3/0/0/0.

Proposed resolution:

Change 27.10.8.5 [path.itr]:

(4.4) — *dot* an empty element, if one or more trailing non-root *slash* characters are present.

Also see LWG 2665 remove_filename() post condition is incorrect

Although not a NB comment, LWG 2665 needs to be coordinated with this set of comments, particularly US-52. It is mentioned here as a reminder to the LWG.

Other comments

US 31; 27.10 [filesystems]: Everything is defined in terms of one implicit host system
Status: Reject

That's by design, aimed at achieving portable syntax and portable behavior. And that objective has been largely achieved in practical real-world use for many years. There is no proposed wording, and without proposed wording there is no consensus for change. Will re-open if a paper or issue appears with proposed wording.

US 32; 27.10.2.1 [fs.conform.9945] ¶3: Meaning of POSIX conformance unclear
Status: Accept with modifications

The comment requests "Clarify that ¶2 governs and an error must be reported in such cases".

Change 27.10.2.1 [fs.conform.9945]:

> Implementations are not required to provide behavior that is not supported by a particular file system. [*Example:* The FAT file system used by some memory cards, camera memory, and floppy disks does not support hard links, symlinks, and many other features of more capable file systems, so implementations are not required to support those features on the FAT file system but instead are required to report an error as described above. —*end example*]

Jonathan Wakely may supply improved wording. The above change, however, is probably sufficient to resolve the NB comment.

US 33; 27.10.4.2 [fs.def.canonical.path]: Definition of *canonical path* problematic
Status: Accept

The comment appears to be correct in that the definition of canonical path is not used in the WP.

Remove 27.10.4.2 [fs.def.canonical.path]:

> **canonical path**
> An absolute path that has no elements that are symbolic links, and no dot or dot-dot elements (27.10.8.1).

US 34; 27.10.4.5 [fs.def.filesystem]: Are there attributes of a file that are not an aspect of the file system?
Status: Accept with modifications

Change 27.10.4.5 [fs.def.filesystem]:

> A collection of files and certain of their attributes.

US 35; 27.10.4.6 [fs.def.race]: What synchronization is required to avoid a file system race?
Status: Reject

27.10.4 Terms and definitions [fs.definitions] is not the proper place to describe file system race behavior. 27.10.2.3 File system race behavior [fs.race.behavior] would be the proper place. There is no proposed wording, and without proposed wording there is no consensus for change. Will re-open if a paper or issue appears with proposed wording.

US 36; 27.10.4.9 [fs.def.link]: Symbolic links themselves are attached to a directory via (hard) links
Status: Accept with modifications

The SG provides wording, taken from POSIX definitions 3.130, Directory Entry (or Link).

Davis Herring plans to open an issue on a related concern that does not affect the resolution of this NB comment below.

Change 27.10.4.9 [fs.def.link]:

> **link**
> A directory entry that associates a filename with a file. A link is either a hard link (27.10.4.8) or a symbolic

link (27.10.4.21). An object that associates a filename with a file. Several links can associate names with the same file.

US 40; 27.10.4.15 [fs.def.parent]: Not all directories have a parent
Status: Reject

No proposed wording. The definition in question is taken directly from the POSIX §3.268 *Parent Directory* definition.

US 43; 27.10.5 [fs.req] ¶4: Concerns about *encoded character types*
Status: Accept with modifications

Change 27.10.5 Requirements [fs.req] ¶4:

[*Note:* Use of an encoded character type implies an associated character set and encoding. Since `signed char` and `unsigned char` have no implied character set and encoding, they are not included as permitted types. — *end note*]

Add a sentence to 27.10.8.2.2 path type and encoding conversions [path.type.cvt]:

If the encoding being converted to has no representation for source characters, the resulting converted characters, if any, are unspecified. Implementations are strongly encouraged not to mutate member function arguments if already of type `path::value_type`. *Billy O'Neal suggested this addition and Beman Dawes supports it, but it has not been reviewed by the rest of the filesystem small group.*

US 44; 27.10.8 [class.path]: Definition of path in terms of a string requires leaky abstraction
Status: Accept with modifications

Proposed wording, including a note, has been applied to LWG 2734 to partially resolve US 44.

The portion of this comment not resolved by LWG 2734 is resolved by US-74/CA-3

Beman Dawes will check 2734, and coordinate with Marshall Clow if needed, or add a comment to 2734

US 45; 27.10.8.1 [path.generic]: Generic format portability compromised by unspecified *root-name*
Status: Accept with modifications

Resolved by US-73/CA-2

US 46; 27.10.8.1 [path.generic]: *filename* can be empty so productions for *relative-path* are redundant
Status: Accept with modifications

Change [path.generic]:

```
relative-path:
  filename
  relative-path directory-separator
  relative-path directory-separator filename
```
an empty path *Note to editor: this line is deliberately not in code font*

...

name:
A non-empty sequence of characters other than directory-separator characters.

Jonathan Wakely supplied the above wording. It has been reviewed by Beman Dawes, but not the rest of the small group.

US 48; 27.10.8.1 [path.generic] ¶1: Multiple separators are often meaningful in a *root-name*
Status: Accept with modifications

Change 27.10.8.1 [path.generic] ¶1:

Except in *root-name*, mMultiple successive *directory-separator* characters are considered to be the same as one *directory-separator* character.

US 49; 27.10.8.2.2 [path.type.cvt]: What does “method of conversion method” mean?
Status: Editorial

Fixed in the post-Issaquah working paper.

US 55; 27.10.8.4.5 [path.modifiers] ¶11: `replace_extension()`'s use of `path` as parameter is inappropriate
Status: Accept with modifications

The SG believes `path` is the appropriate parameter type, but that the standard needs to clarify class `path` usage.

Change 27.10.8 [class.path] ¶1:

> An object of class path represents a path (27.10.4.17) and contains a pathname (27.10.4.18). Such an object is concerned only with the lexical and syntactic aspects of a path. The path does not necessarily exist in external storage, and the pathname is not necessarily valid for the current operating system or for a particular file system.
>
> [*Note:* Class path is used to support the differences between the string types used by different operating systems to represent pathnames, and to perform conversions between encodings when necessary. — *end note*]

US 56; 27.10.8.4.5 [path.modifiers] ¶11.2: Remove `replace_extension()`'s conditional addition of period
Status: Reject

Nico presented this to the Library Evolution Working group in Issaquah, including a number of examples (See Nico's *Filesystem: Filename Issues* slides on the Issaquah LEWG wiki.) There was no consensus for change; the vote was 3/3/3/7/2. See the Issaquah LEWG notes.

US 57; 27.10.8.4.8 [path.compare] ¶2: On Windows, absolute paths will sort in among relative paths
Status: Reject

Insufficient motivation for change.

US 58; 27.10.8.4.9 [path.decompose] ¶5: `parent_path()` behavior for root paths is useless
Status: Accept with modifications

Change [path.decompose]:

```
path parent_path() const;
```

-?- Let `parentEnd` be an instance of `path::iterator`, initialized as follows:

- if `has_relative_path()`, from `--end()`
- otherwise, from `end()`

-5- *Returns:* `(empty() || begin() == --end()) ? path() :` *pp*, where *pp* is a `path` constructed as if by starting with an empty `path` and successively applying `operator/=` for each element in the range `[begin(), --end()parentEnd)`.

US 59; 27.10.8.4.9 [path.decompose] ¶6: `filename()` returning `path` for single path components is bizarre
Status: Reject

`path` is filesystem's vocabulary type for both full paths and for path components, in the same sense that `std::string` might be used as the vocabulary type for both sentences and the words in the sentences. See US 55 for further discussion.

US 61; 27.10.8.4.9 [path.decompose] ¶8: Leading dots in `filename()` should not begin an extension
Status: Accept with modifications

Nico Josuttis presented this to the Library Evolution Working group in Issaquah, including a number of examples (See Nico's *Filesystem: Filename Issues* slides on the Issaquah LEWG wiki.) Support was unanimous. See the Issaquah LEWG notes.

Nico presented the following cases; the only change from the TS is for the `".profile"` case:

path p	***p.stem()***	***p.extension()***
`"/foo/bar.txt"`	`"bar"`	`".txt"`
`".profile"`		`".profile"` `""`

	"" ".profile"	
".profile.old"	".profile"	".old"
"..abc"	"..abc"	""
"...abc"	"...abc"	""
"abc..def"	"abc."	".def"
"abc...def"	"abc.."	".def"
"abc."	"abc"	"."
"abc.."	"abc."	"."
"abc.d."	"abc.d"	"."
".."	".."	""
"."	"."	""

Change 27.10.8.4.9 [path.decompose] ¶ 8:

```
path stem() const;
```

Returns: if `filename()` contains a period other than at the beginning and but does not consist solely of one or two periods, returns the substring of `filename()` starting at its beginning and ending with the character before the last period. Otherwise, returns `filename()`.

Change 27.10.8.4.9 [path.decompose] ¶ 10:

```
path extension() const;
```

Returns: if `filename()` contains a period other than at the beginning and but does not consist solely of one or two periods, returns the substring of `filename()` starting at the rightmost period and for the remainder of the path. Otherwise, returns an empty `path` object.

Beman Dawes supplied the above wording per the LEWG vote. It has not been reviewed by the small group.

US 62; 27.10.8.4.9 [path.decompose] ¶13 (was ¶11): It is important that `stem()+extension()==filename()`
Status: Reject

Yes, it is important. But the note in ¶13 already says exactly that, so the question becomes "Should we make that part of the note normative?"

Saying that normatively would be a second way of saying what is already specified behavior, and saying the same normative thing twice is not good standardese. No proposed wording.

US 63; 27.10.8.4.11 [path.gen] ¶1: `lexically_normal()` inconsistently treats trailing "/" but not "/.." as directory
Status: Accept with modifications

See US-37 and the change to [path.gen] in US-74/CA-3.

US 185; 27.10.7 [fs.err.report]: Fold `error_code` and non-`error_code` signatures into one signature
Status: Reject

Does not handle one signature being `noexcept(true)` and the other `noexcept(false)`, and many other problems. Even the submitter is no longer in favor. See LWG mailing list thread "[filesystem] US-185 Eliminating dual signatures for operational functions", October 25, 2016.

FI 14; 27.10.12.3 [directory_entry.obs]: `directory_entry` comparisons are members
Status: Reject

This is LWG issue 2761, which has been closed as NAD.

US 73, CA 2; 27.10.8.1 [path.generic]: *root-name* is effectively implementation defined
Status: Accept with modifications

Change [path.generic]:

root-name:

An operating system dependent name that identifies the starting location for absolute paths. If the operating system does not define at least one *root-name*, then the implementation defines a *root-name*. Implementations are permitted to define additional *root-names*. [*Note:* Many operating systems define a name beginning with two *directory-separator* characters as a *root-name* that identifies network or other resource locations. Some operating systems define a single letter followed by a colon as a drive specifier – a *root-name* identifying a specific device such as a disk drive. —*end note*]

Add a new paragraph at the end of [path.generic]:

If *root-name* is otherwise ambiguous, the possibility with the longest sequence of characters is chosen. [Note: on a POSIX-like operating system, it is impossible to have a *root-name* and a *relative-path* without an intervening *root-directory element*. -- end note]

US 74, CA 3; 27.10.8 [class.path]: The term "pathname" is ambiguous in some contexts
Status: Accept with modifications

Davis Herring supplied the wording at the request of the filesystem small group in Issaquah. The wording has been reviewed by Beman Dawes but not the rest of the small group.

This NB comment is a showstopper for C++17. These specification improvements enable the resolution of many other filesystem NB comments.

Discussion:

Many operations on `filesystem::path` are defined in terms of its exposition-only `pathname` member, which is defined implicitly by `path::native()` as the native format path. However, the native format need not be compatible with lexical operations like, for example, `path::root_name()` and `path::extension()`.

This same concern was also raised by P0430R0 Section 2.1, which provided additional rationale.

The proposed resolution below clarifies the distinct role played by the `+= operator` as mentioned in US 44 and addresses US 74/CA 3 (with a more extensive resolution).

Proposed resolution:

Remove In the [class.path] synopsis, remove the private (exposition-only) member from class path:

```
private:
  string_type pathstring; // exposition only
```

Change [path.fmt.cvt]:

Option A; from Davis:

[*Note:* The format conversions described in this section are not applied on POSIX- or Windows-based operating systems because on these systems:

— The generic format is acceptable as a native path.
— There is no need to distinguish between native format and generic format in function arguments.
— Paths for regular files and paths for directories share the same syntax.

—end note]

Option B; from Beman:

[*Note:* The format conversions described in this section are usually not applied on POSIX- or Windows-based operating systems because on these systems:

— The generic format is acceptable as a native path.
— There is usually no need to distinguish between native format and generic format in function arguments.
— Paths for regular files and paths for directories share the same syntax.

Even on these operating systems, the format conversions may be applied for pathnames with implementation-defined *root-name*s ([path.generic]).

—end note]

We need to choose between Option A or Option B.

Several functions are defined to accept *detected-format* character sequence arguments that represent paths using either the generic pathname format grammar ([path.generic]) or the native pathname format ([fs.def.native]). Such an argument is taken to be in the generic format if and only if it matches the generic format but is not acceptable to the operating system as a native path.

Function arguments that take character sequences representing paths may use the generic pathname format grammar (27.10.8.1) or the native pathname format (27.10.4.11). If and only if such arguments are in the generic format and the generic format is not acceptable to the operating system as a native path, conversion to native format shall be performed during the processing of the argument.

[Note: Some operating systems may have no unambiguous way to distinguish between native format and generic format arguments. This is by design as it simplifies use for operating systems that do not require disambiguation. An implementation for an operating system where disambiguation is required is permitted to distinguish between the formats. —end note]

Beman says his code font markup of text from Davis in the following paragraph is almost certainly wrong:

- *Would s/Define G(n) and N(g) as the functions/Let G and N be the implementation's functions/ be clearer?*
- *Are some of the `'='` supposed to be `'=='`?*

Pathnames are converted as needed between the generic and native formats in an operating-system-dependent manner. Define `G(n)` and `N(g)` as the functions that convert to the generic and native formats respectively. If `g=G(n)` for some `n`, then `G(N(g))=g`; if `n=N(g)` for some `g`, then `N(G(n))=n`. [*Note:* Neither `G` nor `N` need be invertible. —*end note*]

If the native format requires paths for regular files to be formatted differently from paths for directories, the path shall be treated as a directory path if its last element is a directory-separator, otherwise it shall be treated as a path to a regular file.

When a path is constructed from or is assigned a single representation separate from any path, the other representation is selected by the appropriate conversion function (`G` or `N`).

When the (new) value `p` of one representation of a path is derived from the representation of that or another path, a value `q` is chosen for the other representation. The value `q` converts to `p` (by `G` or `N` as appropriate) if any such value does so; `q` is otherwise unspecified. [*Note:* If `q` is the result of converting any path at all, it is the result of converting `p`. —*end note*]

[SG note: P0430R1 (for US 75/CA 4) adds support for explicitly specifying the format to use, at least in the constructor.]

Change [path.construct]:

```
path(const path& p);
path(path&& p) noexcept;
```

> *Effects:* Constructs an object of class `path` having the same native and generic format pathnames as `p` with `pathstring` having the original value of `p.pathstring`. In the second form, `p` is left in a valid but unspecified state.

```
path(string_type&& source);
```

> *Effects:* Constructs an object of class `path` for which the pathname in the detected-format of `source` has with `pathstring` having the original value of `source` ([path.fmt.cvt]). `source` is left in a valid but unspecified state.

```
template <class Source>
path(const Source& source);
template <class InputIterator>
path(InputIterator first, InputIterator last);
```

> *Effects:* Let `s` be the effective range of `source` ([path.req]) or the range `[first, last)`, with the encoding converted if required ([path.cvt]). Finds the detected-format of `s` ([path.fmt.cvt]) and constructs an object of class `path` for which the pathname in that format is `s`. Constructs an object of class `path`, storing the effective range of source (27.10.8.3) or the range `[first, last)` in `pathstring`, converting format and encoding if required (27.10.8.2).

```
template <class Source>
path(const Source& source, const locale& loc);
template <class InputIterator>
path(InputIterator first, InputIterator last, const locale& loc);
```

> *Requires:* The value type of `Source` and `InputIterator` is `char`.

Effects: Constructs an object of class `path`, storing the effective range of source or the range `[first, last)` in `pathstring`, after converting format if required and Let `s` be the effective range of `source` or the range `[first, last)`, after converting the encoding as follows:

— If `value_type` is `wchar_t`, converts to the native wide encoding (27.10.4.10) using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`.

— Otherwise a conversion is performed using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`, and then a second conversion to the current narrow encoding.

Finds the detected-format of `s` ([path.fmt.cvt]) and constructs an object of class `path` for which the pathname in that format is `s`.

In the example after [path.fmt.cvt] ¶ 7.2, change:

For POSIX-based operating systems, the path is constructed by first using `latin1_facet` to convert ISO/IEC 8859-1 encoded `latin1_string` to a wide character string in the native wide encoding (27.10.4.10). The resulting wide string is then converted to a narrow character `pathstring` pathname string in the current native narrow encoding. If the native wide encoding is UTF-16 or UTF-32, and the current native narrow encoding is UTF-8, all of the characters in the ISO/IEC 8859-1 character set will be converted to their Unicode representation, but for other native narrow encodings some characters may have no representation.

For Windows-based operating systems, the path is constructed by using `latin1_facet` to convert ISO/IEC 8859-1 encoded `latin1_string` to a UTF-16 encoded wide character `pathstring` pathname string. All of the characters in the ISO/IEC 8859-1 character set will be converted to their Unicode representation.

Change [path.assign]:

```
path& operator=(const path& p);
```

Effects: If `*this` and `p` are the same object, has no effect. Otherwise, sets the native and generic format pathnames to those of `p` modifies `pathstring` to have the original value of `p.pathstring`.

Returns: `*this`.

```
path& operator=(path&& p) noexcept;
```

Effects: If `*this` and `p` are the same object, has no effect. Otherwise, sets the native and generic format pathnames to those of `p` modifies `pathstring` to have the original value of `p.pathstring`. `p` is left in a valid but unspecified state. [*Note:* A valid implementation is `swap(p)`. —*end note*]

Returns: `*this`.

```
path& operator=(string_type&& source);
path& assign(string_type&& source);
```

Effects: Sets the native and generic format pathnames to those of `source` Modifies `pathstring` to have the original value of `source`. `source` is left in a valid but unspecified state.

Returns: `*this`.

```
template <class Source>
path& operator=(const Source& source);
template <class Source>
path& assign(const Source& source);
template <class InputIterator>
path& assign(InputIterator first, InputIterator last);
```

Effects: Let `s` be the effective range of `source` ([path.req]) or the range `[first, last)`, with the encoding converted if required ([path.cvt]). Finds the detected-format of `s` and sets the pathname in that format to `s` ([path.fmt.cvt]). Stores the effective range of `source` (27.10.8.3) or the range `[first, last)` in `pathstring`, converting format and encoding if required (27.10.8.2).

Returns: `*this`.

Change [path.append] ¶2 & 3:

```
path& operator/=(const path& p);
```

Effects: If equivalent to `has_filename() || !has_root_directory() && is_absolute()`, then appends `path::preferred_separator` to the generic format pathname. [*Example:* `path("//host")/"foo"` and

`path("//host/")/"foo"` both equal `"//host/foo"`. —*end example*]

Appends `path::preferred_separator` to `pathstring` unless:

— an added *directory-separator* would be redundant, or

— an added *directory-separator* would change a relative path into an absolute path [*Note:* An empty path is relative.—*end note*] , or

— `p.empty()` is `true`, or

— `*p.native().cbegin()` is a directory-separator.

Then appends `p.native()` to the native format pathname `pathstring`.

Returns: `*this`.

[SG note: this proposed wording depends on LWG 2732 and the recently proposed definition of has_filename() and will change again to address LWG 2664 and US 77/CA 6. It already, simply by ignoring p entirely until such time, addresses US 51.]

Change [path.modifiers]:

```
path& make_preferred();
```

Effects: Each *directory-separator* is converted to *preferred-separator* in the generic format pathname.

[SG note: the existing definition of remove_filename() is broken; it is expected to be rewritten.]

Change [path.modifiers]:

```
path& replace_extension(const path& replacement = path());
```

Effects: `pathstring` (the stored path) is modified as follows:

— Any existing `extension()`(27.10.8.4.9) is removed from the generic format pathname stored path, then

— If `replacement` is not empty and does not begin with a *dot* character, a *dot* character is appended to the generic format pathname stored path, then

— `operator+=(replacement);` `replacement` is concatenated to the stored path.

Returns `*this`.

Change [path.modifiers]:

```
void swap(path& rhs) noexcept;
```

Effects: Swaps the contents (in all formats) of the two paths `pathstring` and `rhs.pathstring`.

Change [path.native.obs]:

```
const string_type& native() const noexcept;
```

Returns: The native format pathname `pathstring`.

```
const value_type* c_str() const noexcept;
```

Returns: Equivalent to `native().c_str()` `pathstring.c_str()`.

```
operator string_type() const;
```

Returns: `native()` `pathstring`.

[*Note:* Conversion to string_type is provided so that an object of class path can be given as an argument to existing standard library file stream constructors and open functions. —*end note*]

```
template <class EcharT, class traits = char_traits<EcharT>,
          class Allocator = allocator<EcharT>>
  basic_string<EcharT, traits, Allocator>
    string(const Allocator& a = Allocator()) const;
```

Returns: native() pathstring.

Remarks: All memory allocation, including for the return value, shall be performed by a. Conversion, if any, is specified by 27.10.8.2.

Change [path.generic.obs]:

```
template <class EcharT, class traits = char_traits<EcharT>,
          class Allocator = allocator<EcharT>>
  basic_string<EcharT, traits, Allocator>
    generic_string(const Allocator& a = Allocator()) const;
```

Returns: The generic format pathname pathstring, reformatted according to the generic pathname format (27.10.8.1).

Remarks: All memory allocation, including for the return value, shall be performed by a. Conversion, if any, is specified by 27.10.8.2.

```
std::string generic_string() const;
std::wstring generic_wstring() const;
std::string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

Returns: The generic format pathname pathstring, reformatted according to the generic pathname format (27.10.8.1).

Remarks: Conversion, if any, is specified by 27.10.8.2. The encoding of the string returned by generic_u8string() is always UTF-8.

Change [path.decompose]:

```
path root_name() const;
```

Returns: root-name, if the generic format pathname pathstring includes *root-name*, otherwise path().

```
path root_directory() const;
```

Returns: root-directory, if the generic format pathname pathstring includes *root-directory*, otherwise path().

...

```
path relative_path() const;
```

Returns: A path composed from the generic format pathname pathstring, if !empty(), beginning with the first *filename* after *root-path*. Otherwise, path().

...

```
path filename() const;
```

Returns: relative_path().empty() ? path() : *--end().

[*Example:*

```
std::cout << path("/foo/bar.txt").filename(); // outputs "bar.txt"
std::cout << path("/foo/bar").filename();     // outputs "bar"
std::cout << path("/foo/bar/").filename();    // outputs ""
std::cout << path("/").filename();            // outputs "/"
std::cout << path("//host").filename();       // outputs ""
std::cout << path(".").filename();            // outputs "."
std::cout << path("..").filename();           // outputs ".."
```

—*end example*]

```
path stem() const;
```

Returns: if filename() contains a period but does not consist solely of one or two periods, returns the substring of filename() starting at its beginning and ending with the character before the last period. Otherwise, returns filename().

Returns: Let f be the generic format pathname for filename(). Returns a path whose generic format pathname is

- `f`, if it contains no periods other than a leading period or consists solely of one or two periods;

- otherwise, the prefix of `f` ending before its last period.

Beman Dawes inserted "other than a leading period" to reflect LEWG guidance from Issaquah. That wording has not been reviewed by the filesystem small group.

[Example:

```
std::cout << path("/foo/bar.txt").stem(); // outputs "bar"
path p = "foo.bar.baz.tar";
for (; !p.extension().empty(); p = p.stem())
std::cout << p.extension() << '\n';
// outputs: .tar
// .baz
// .bar
```

—end example]

```
path extension() const;
```

Returns: a path whose generic format pathname is the suffix of the generic format pathname for `filename()` not included in `stem()`. if `filename()` contains a period but does not consist solely of one or two periods, returns the substring of `filename()` starting at the rightmost period and for the remainder of the path. Otherwise, returns an empty path object.

Remarks: Implementations are permitted to define additional behavior for file systems which append additional elements to extensions, such as alternate data streams or partitioned dataset names.

[*Example:*

```
std::cout << path("/foo/bar.txt").extension(); // outputs ".txt"
std::cout << path("/foo/bar").extension();     // outputs ""
```

—end example]

[*Note:* The period is included in the return value so that it is possible to distinguish between no extension and an empty extension. Also note that for a path `p`, `p.stem()+p.extension() == p.filename()`. *—end note*]

Change [path.query]:

```
bool empty() const noexcept;
```

Returns: `true` if the generic format pathname is empty, else `false` `pathstring.empty()`.

...

```
bool is_absolute() const;
```

Returns: `true` if the native format pathname `pathstring` contains an absolute path (27.10.4.1), else false.

[Example: `path("/").is_absolute()` is true for POSIX-based operating systems, and false for Windows-based operating systems. —end example]

Change [path.gen]:

```
path lexically_normal() const;
```

Returns: a path whose generic format pathname is the normal form ([fs.def.normal.form]) of the generic format pathname of `*this` `*this` in normal form ([fs.def.normal.form]).

[*Example:*

```
assert(path("foo/./bar/..").lexically_normal() == "foo");
assert(path("foo/.///bar/../").lexically_normal() == "foo/.");
```

The above assertions will succeed. The second example ends with a current directory (dot) element appended to support operating systems that use different syntax for directory names and regular file names.

On Windows, the returned path's directory-separator characters will be backslashes rather than slashes, but that does not affect path equality. *—end example*]

Change [path.itr]:

> Path iterators iterate over the elements of the generic format pathname `pathstring` in the generic format (27.10.8.1).
>
> A `path::iterator` is a constant iterator satisfying all the requirements of a bidirectional iterator (24.2.6) except that, for dereferenceable iterators `a` and `b` of type `path::iterator` with `a == b`, there is no requirement that `*a` and `*b` are bound to the same object. Its `value_type` is `path`.
>
> Calling any non-const member function of a path object invalidates all iterators referring to elements of that object.
>
> For the elements of the generic format pathname `pathstring` in the generic format, the forward traversal order is as follows:
>
> ...

Change [fs.op.canonical]:

```
path canonical(const path& p, error_code& ec);
path canonical(const path& p, const path& base, error_code& ec);
```

> *Effects:* Converts `p`, which must exist, to an absolute path that has no symbolic link, *dot*, or *dot-dot* elements in its generic format pathname ".", or ".." elements.
>
> ...

Change [fs.op.current_path]:

```
path current_path();
path current_path(error_code& ec);
```

> *Returns:* The absolute path of the current working directory, obtained in the native format as if by POSIX `getcwd()`. The signature with argument `ec` returns `path()` if an error occurs.
>
> ...

Rationale:

The lexical operation specifications leave some room for variation among implementations, but not as much as may be required for native formats; further relaxation would compromise the utility of the operations for portable, predictable path manipulation. Instead, we specify a usable but flexible mapping between the two formats and define each operation in terms of one (or, occasionally, both).

The second bullet item in the first note in [path.fmt.cvt], which says for POSIX- or Windows-based operating systems "There is no need to distinguish between native format and generic format in function arguments.", may not be true on Windows even if the other two bullet items are true.

US 75, CA 4; 27.10.8.4.1 [path.construct]: Extra flag in path constructors is needed
Status: Accept with modifications

> See P0430R1 Section 2.2 for wording.

US 76, CA 5; 27.10.8.1 [path.generic]: *root-name* definition is over-specified
Status: Accept with modifications

> See P0430R1 Section 2.3.1 for wording.

US 77, CA 6; 27.10.8.4.3 [path.append]: `operator/` and other appends not useful if arg has *root-name*
Status: Accept with modifications

> See P0430R1 Section 2.3.2 for wording.

US 78, CA 7; 27.10.15.1 [fs.op.absolute]: Member `absolute()` in 27.10.4.1 is overspecified for non-POSIX-like O/S
Status: Accept with modifications

> See P0430R1 Section 2.4.1 for wording.

US 79, CA 8; Several: Some operation functions are overspecified for implementation-defined file types
Status: Accept with modifications

See P0430R1 Section 2.4.2 for wording.

NB comments submitted as editorial

There was concern that some of these might involve substantive changes, so the SG processed them as ordinary technical comments.

US 38; 27.10.4.13 [fs.def.symlink]: Duplicates §17.3.16
Status: Accept

Project editor will make change.

US 39; 27.10.4.15 [fs.def.parent]: Remove note: Dot and dot-dot are not directories
Status: Accept

Project editor will make change.

US 41; 27.10.4.16 [fs.def.parent.other]: The term “parent directory” for a (non-directory) file is unusual
Status: Reject

Insufficient motivation for change. "parent directory" is the term we use to describe the containing directory; we're (already) following POSIX there, even though the POSIX definition is a bit weird.

US 42; 27.10.4.21 [fs.def.symlink]: Pathname resolution does not always resolve a symlink
Status: Reject

There is an apparent corner case contradiction between the POSIX definition and other places in the POSIX standard. The [fs.def.symlink] definition is identical to the POSIX 3.381 Symbolic Link definition () and the SG prefers to keep it that way.

US 47; 27.10.8.1 [path.generic]: “.” and “..” already match the name production
Status: Accept with modifications

This is not editorial, but we know the fix. Davis Herring to supply updated proposed wording.

US 50; 27.10.8.3 [path.req]: `path` requirements ¶ 1.4 largely redundant with ¶ 1.3
Status: Reject

Beman Dawes to open an issue to expose this concern to additional experts. This is not a priority issue for shipping C++17.

Late comments from P0489R0

Quoting P0489R0, "These comments were not submitted in time to be registered as National Body Comments, but should be considered as possible issues against SC 22, N3151, ISO/IEC CD 14882."

Late comments 15-47 apply to 27.10 [filesystems]. Except for as noted below, the submitter of these late comments has agreed to open LWG issues for any late comments that are still a concern.

Late 15; 27.10.8.4.11 ¶3.1: A “mismatched element” cannot be equal to an iterator

Late 16; 27.10.8.4.11 ¶3.3.1: `lexically_relative("..\\foo")` produces nonsense.

Late 17; 27.10.8.4.11 ¶3.3.2: Behavior not always appropriate

Late 18; 27.10.8.4.11 ¶4: Behavior wrong in case corresponding to comment Late17

Late 19; 27.10.8.5 ¶2: It is surprising that a path is a container of paths

Recommend NAD. See US 55, US 59.

Late 20; 27.10.11 ¶1: Anemic status structure

Late 21; 27.10.12: `directory_entry` is just a trivial wrapper for `path`

Subsumed by LWG 2663 and 2677

Late 22; 27.10.13 ¶6: `directory_iterator` cumbersome to assemble full path with iterator's directory

Late 23; 27.10.14.1 ¶28: `disable_recursion_pending()` name is ugly implementation detail

Late 24; 27.10.15.1 ¶1: `absolute()` can produce nonsense result

Late 25; 27.10.15.4 ¶4.2: What attributes does `copy_file()` copy?

Late 26; 27.10.15.6 ¶5: `create_directories()` complexity assumes syscalls take constant time

Late 27; 27.10.15.7 ¶5: `create_directory()` specification prevents sensible security measures

Late 28; 27.10.15.10: `create_symlink()` might misbehave if `to` is a directory

Late 29; 27.10.15.13: Access to file ID is superior to `equivalent()`

Late 30; 27.10.15.13 ¶3: `equivalent()`'s `s1==s2` check is ill-formed and could race

Late 31; 27.10.15.13 ¶5: Surprising that `equivalent()`'s error_code overload can throw

Late 32; 27.10.15.13 ¶5 : `equivalent()` should not reject special files outright

Late 33; 27.10.15.21: `is_other()` result surprising

Late 34; 27.10.15.25 ¶5: Could `last_write_time()` guarantee the error direction

Recommend submitting a paper that researches whether this is in fact possible. If it is possible, providing proposed wording would be helpful.

Late 35; 27.10.15.26: Why is there no way to read permissions?

Recommend NAD. They are available in `file_status`. If anyone thinks a separate function would be useful, they should submit an issue or a paper.

Late 36; 27.10.15.26: The `permissions() error_code` overload should be `noexcept`
Status: Accept

Change 27.10.15.26 [fs.op.permissions] and 27.10.6 Header <filesystem> synopsis [fs.filesystem.syn]:

```
void permissions(const path& p, perms prms, error_code& ec) noexcept;
```

Late 37; 27.10.15.26 ¶2:`permissions()` actions should be separate parameter
Status: Accept with modifications

Beman Dawes supplied the wording, which has been reviewed by Davis but not others. Beman comments: Matt Austern pointed this out, with the same suggested fix, prior to the TS. It got dropped on the floor in the TS shipping rush and has never been fixed. It is an embarrassment that needs fixing before shipping C++17.

Change [fs.filesystem.syn]:

```
// 27.10.10, enumerations
enum class file_type;
enum class perms;
enum class perm_options;
enum class copy_options;
enum class directory_options;

...

void permissions(const path& p, perms prms, perm_options opts=perm_options::replace);
void permissions(const path& p, perms prms, error_code& ec);
void permissions(const path& p, perms prms, perm_options opts, error_code& ec);
```

Strike the following rows in [fs.enum] Table nnn — Enum class `perms`:

- *`add_perms`*
- *`remove_perms`*
- *`symlink_nofollow`*

Insert a new sub-section after [enum.perms]:

> ***27.10.n.n*** **Enum class `perm_options` [enum.perm_options]**
>
> The `enum class` type `perm_options` is a bitmask type (bitmask.types) that specifies bitmask constants used to control the semantics of permissions operations.
>
> Table *nnn* — Enum class `perm_options`
>
Name	**Meaning**
`replace`	(Default) `permissions()` shall replace the file's permission bits with the `perm` argument's permission bits.
`add`	`permissions()` shall bitwise *or* the `perm` argument's permission bits to the file's current permission bits.
`remove`	`permissions()` shall bitwise *and* the complement of the `perm` argument's permission bits to the file's current permission bits.
`nofollow`	`permissions()` shall change the permissions of a symbolic link itself rather than the permissions of the file the link resolves to. `nofollow` is a bit-mask value.

Change [fs.op.permissions]:

```
void permissions(const path& p, perms prms, perm_options opts=perm_opts::replace);
void permissions(const path& p, perms prms, error_code& ec);
void permissions(const path& p, perms prms, perm_options opts, error_code& ec);
```

> *Requires:* `!((prms & perms::add_perms) != perms::none && (prms & perms::remove_perms) != perms::none)`. One and only one of the `perm_options` constants `replace`, `add`, or `remove` is present in `opts`.
>
> *Remarks:* The second signature behaves as if it had an additional argument `perm_options opts` with a value of `perm_options::replace`.
>
> *Effects:* Applies the effective permissions bits from `prms` to the file `p` resolves to, or if that file is a symbolic link and `symlink_nofollow` is not set in `prms`, the file that it points to, as if by POSIX `fchmodat()`. The effective permission bits are determined as specified in Table 127, where `s` is the result of `(prms & perms::symlink_nofollow) != perms::none ? symlink_status(p) : status(p)`.
>
> *Effects:* Applies the action specified by `opts` to the file `p` resolves to, or to file `p` itself if `p` is a symbolic link and `perm_options::nofollow` is set in `opts`. The action is applied as if by POSIX `fchmodat()`.
>
> [*Note:* Conceptually permissions are viewed as bits, but the actual implementation may use some other mechanism. —*end note*]
>
> *Throws:* As specified in 27.10.7.

Late 38; 27.10.15.26 ¶2: Unimplementable implication that `permissions()` is atomic

Late 39; 27.10.15.29 ¶3: Suprising `relative()` behavior for symlinks

Late 40; 27.10.15.31 ¶1: `remove_all()` unclear for `symlink/`

Late 41; 27.10.15.31 ¶3: Would be useful for `remove_all()` to report successful removal count on error

Late 42; 27.10.15.33 ¶1: `resize_file()` *Postcondition* missing argument
Status: Editorial

> Fixed in the post-Issaquah working paper

Late 43; 27.10.15.33 ¶3: Add POSIX `ftruncate()` equivalent function

Recommend opening issue. Such a feature might be useful in 27.9 [file.streams] for C++2n.

Late 44; 27.10.15.38: `system_complete()` name inconsistent with similar functions in subclause

Late 45; 27.10.15.38 ¶6: Remove `system_complete()` unless it can be made reliable

Late 46; 27.10.15.40 ¶2: Question about `weakly_canonical()` *Effects*

Late 47; 27.10.15.40 ¶4: `weakly_canonical()` suggested caching is incorrect
